



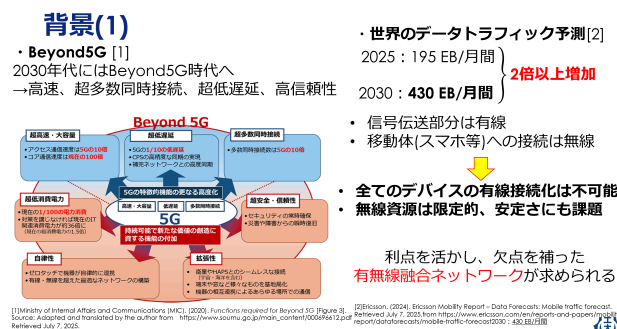
# R8研究進捗報告(2026.06.03)

|        |                                                                                          |
|--------|------------------------------------------------------------------------------------------|
| 報告者    |  市川慧騎   |
| 研究題目   | 固定無線と有線ネットワークを統合制御するネットワーク制御技術の研究                                                        |
| 作成日時   | 2026年4月23日 22:11                                                                         |
| 最終更新日時 | 2026年6月3日 17:46                                                                          |
| 研究トピック | ネットワーク制御                                                                                 |
| 研究者名   |  市川慧騎 |

## • Background , Purpose

### ▼ 背景、課題、解決方法

#### 背景 1：有無線融合ネットワーク



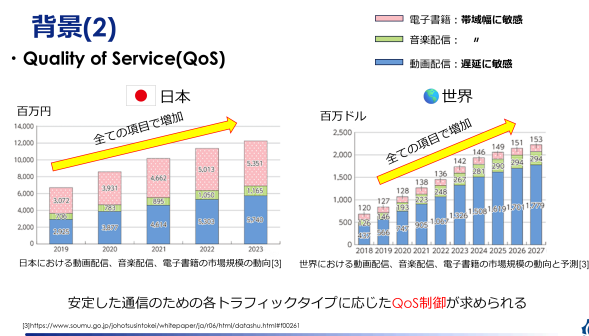
2030年代にBeyond5G時代を迎え、超高速、大規模接続、超低遅延、高信頼性が求められる。また、世界のデータトラフィックは急速に増加しており、月間のモバイルデータトラフィックは2025年の195EBから2030年には2倍以上となる430EBにまで増加すると予想されている。

このように増加するグローバルトラフィックに対して長距離にわたる信号伝送部分は有線接続、スマホなどの移動体には無線接続などのそれぞれの利点を活かす通信が求められている。

しかし、全てのデバイスを有線で接続するのはコストや労力の観点から現実的に不可能である。また全てのデバイスを無線で接続することも限られた無線資源の状況から不可能であり、大気や雨などの環境による影響を受けて安定さにも課題が残っている。

このことから各々の利点を活かし、欠点を補える有無線融合ネットワークが求められる。

## 背景 2：Quality of Service(QoS)



上図は日本と世界の動画配信、音楽配信、電子書籍の市場動向である。桃色が電子書籍、緑色が音楽配信、青色が動画配信を示している。全ての項目について日本と世界ともに市場が成長しており、それに伴ってこれらのデータ通信量も増加することが予測される。

これらのデータにおいてそれぞれ特徴が異なり、リアルタイムの動画配信は遅延が少ないことが重要な一方で、電子書籍はリアルタイム性よりもデータに欠損が無く送られなければならないため帯域幅の大きさが重要となる。

これらの特徴を無視して通信をすると映像から音が遅れて聞こえたり、ファイルデータが欠損して送られてしまう可能性がある。各種のトラフィックに対して安定した通信を確保するためにはサービス品質を担保するQoS制御が必要となる。

## 課題と解決方法

- 課題
  - 全てのデバイスの有線接続化は不可能
  - 無線資源は限定的、安定さにも課題
  - 多様で膨大なトラフィックに対する優先制御が必要



安定した**有無線融合ネットワーク**の構築  
多様で膨大なトラフィックに対する**QoS制御**

- 解決方法
  - 安定した有無線融合ネットワークの構築
    - MPLS : ラベルによる明示的な経路制御、各種サービスのサポート
  - QoS制御
    - DiffServ : 優先度制御、DSCPによりマーキング
    - MPLS : 優先度に基づくルーティング制御(IPルーティングでは不可能)

MPLS : Multi Protocol Label Switching  
DiffServ : Differentiated Services  
DSCP : Differential Service Code Point

図 10-10-10-10

## 課題

以上の背景から問題点として「①全てのデバイスの有線接続化は不可能②無線資源は限定的で安定さにも課題③多様で膨大なトラフィックに対する優先制御が必要」の3つを挙げる。この3つの問題点を解決するための課題として「①安定した有無線融合ネットワークの構築②多様で膨大なトラフィックに対するQoS制御」の2つを設定する。

## 解決方法

①安定した有無線融合ネットワークの構築→MPLS：ラベルによる明示的な経路制御、各種サービスのサポート

②QoS制御→DiffServ：優先度制御 MPLS：優先度に基づくルーティング制御

12/11更新

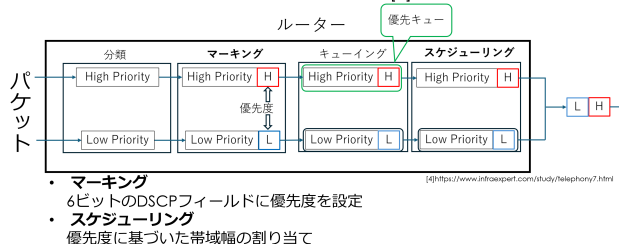
なぜMPLSを選んだのかの記述も入れたい

▼ 原理 説明文を適宜追加していく

- DiffServ

### DiffServ

- パケットの優先度ごとにルーターが処理[4].

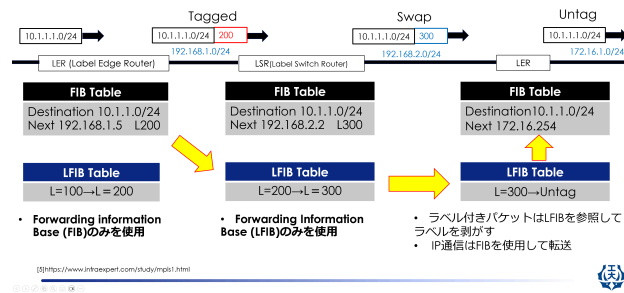


- マーキング  
6ビットのDSCPフィールドに優先度を設定
- スケジューリング  
優先度に基づいた帯域幅の割り当て



- MPLS

## MPLS[5]



- DSCP

## DSCP

- **EF**(Expedited Forwarding), **BE**(Best Effort), **AF**(Assured Forwarding),
- EF: 低欠落、低遅延の**高品質**な転送
- BE: EF、AF以外の転送、**最低**の優先度
- AF: 最小保証帯域幅による4クラス、データ処理優先度による3クラスの12クラスによる転送

|       |       |          |        |        |        |
|-------|-------|----------|--------|--------|--------|
|       |       | 最低保証帯域 大 |        |        |        |
|       |       | Class1   | Class2 | Class3 | Class4 |
| 欠落度 多 | Drop1 | AF11     | AF21   | AF31   | AF41   |
|       | Drop2 | AF12     | AF22   | AF32   | AF42   |
|       | Drop3 | AF13     | AF23   | AF33   | AF43   |

- 参照サイト

QoS - DSCP (Differentiated Services Code Point) とは

QoS - DiffServモデルの処理フローMPLS - MPLSの概要

MPLS - MPLSの概要

## • Keywords

Beyond5G

ネットワーク

有無線融合

QoS

輻輳検知

MPLS



## • What was revealed

やろうとしたこと：ホワイトボックススイッチを使って実験環境を構築、検証

PCとSonicスイッチを繋いでMPLSとQoSの設定を組み、実機検証

### ▼ ホワイトボックススイッチとLinuxPCの接続の備忘録

ホワイトボックススイッチとPC(kannolab)をLAN接続

Win+R→ncpa.cplからイーサネット接続を右クリックしてプロパティを開く

インターネットプロトコルバージョン4を右クリックしてプロパティ

192.168.128.10と255.255.255.0と設定

cmdでipconfigを打ち、イーサネットのIPアドレスが設定したものになっているかを確認

その後、ssh kannolab@192.168.128.33

→いつものPWを入力

```
C:\Users\菅野研究室mousePC2023-1>ssh root@192.168.128.33
The authenticity of host '192.168.128.33 (192.168.128.33)' can't
be established.
RSA key fingerprint is SHA256:W+Sx0aDBAkMZf0zLPheZn0WVWF4uTACTAX
lLZu82Whk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Failed to add the host to the list of known hosts (C:\Users\\350\217\205\351\207\216\347\240\224\347\251\266\345\256\244mousePC2023-1/.ssh/known_hosts).
Debian GNU/Linux 11 \n \l

root@192.168.128.33's password: |
```

左上からeth0で始まり、下がeth1、その下がeth2、右上eth3という感じ

グループごとで速度を通信速度を同じにしないといけない

[https://www.edge-core.com/wp-content/uploads/2025/08/2024-039-AS7326-56X\\_QSG-EN\\_SC\\_TC-R04-20250815.pdf](https://www.edge-core.com/wp-content/uploads/2025/08/2024-039-AS7326-56X_QSG-EN_SC_TC-R04-20250815.pdf)

**Note:** All SFP28 ports (1-48) in the same port group must be set to the same speed (25G or 10G). The default ports speed is 25G.

The switch port groups are as follows:

- 1, 2, 3, 6
- 4, 5, 7, 9
- 8, 10, 11, 12
- 13, 14, 15, 18
- 16, 17, 19, 21
- 20, 22, 23, 24
- 25, 26, 27, 30
- 28, 29, 31, 33
- 32, 34, 35, 36
- 37, 38, 39, 42
- 40, 41, 43, 45
- 44, 46, 47, 48

25Gと100Gのポートに10GのSFPを差し込んでいたためpingが通らなかった

- **ポートの速度を25Gから10G に変更**

```
sudo sed -i 's/"speed": "25000"/"speed" "10000"/g' /etc/sonic/config_db.json
```

- **設定反映**

```
sudo config reload -y
```

- **設定確認**

```
show interface status
```

Linux enp4s0のLANポートとMGMTを接続

LinuxPCのSFP+ポートとスイッチを繋いだ

#### ▼ 光ファイバー

| 項目       | シングルモード (SM / LR)       | マルチモード (MM / SR)         |
|----------|-------------------------|--------------------------|
| 正式名称     | 10GBASE-LR (Long Reach) | 10GBASE-SR (Short Reach) |
| 主な波長     | 1310nm / 1550nm         | 850nm                    |
| ファイバーの色  | 黄色                      | 水色 (または橙色)               |
| 芯径 (コア径) | 約9μm (非常に細い)            | 50μm または 62.5μm (太い)     |
| 通信距離     | 長距離 (10km ~ 40km以上)     | 短距離 (30m ~ 400m程度)       |

|        |                               |                            |
|--------|-------------------------------|----------------------------|
| 主な用途   | 建物間接続、拠点間通信、キャリア網、大規模DCの長距離配線 | 同一ラック内、隣接ラック間、同一フロア内の短距離接続 |
| コスト    | モジュール本体が高価                    | モジュール本体が安価                 |
| 接続の安定性 | 非常に高い                         | 距離や曲げに弱く、減衰しやすい            |

| 確認場所    | シングルモード (SM)    | マルチモード (MM) |
|---------|-----------------|-------------|
| 略称      | LR              | SR          |
| 波長 (nm) | 1310nm / 1550nm | 850nm       |
| レバーの色   | 青               | 黒 (またはベージュ) |
| ケーブルの色  | 黄色              | 水色 (または橙)   |

## ▼ リモート接続

### • Macの場合

ターミナル↓

```
ssh -L 5900:localhost:5900 -N -l kannolab 133.68.41.182
```

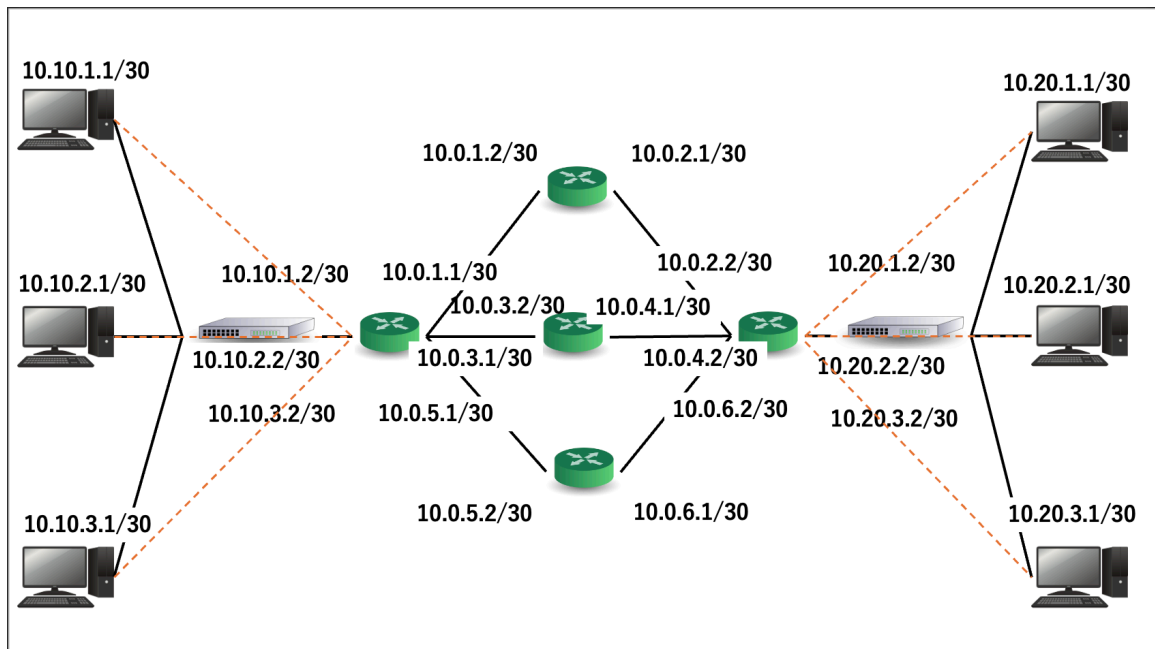
↓command + K

```
vnc://127.0.0.1:5900
```

### • Windowsの場合

```
ssh -L 5900:localhost:5900 -N -l kannolab 133.68.41.182
```

## ▼ 配線



送受信ノード→LERルータ間→MMF(数m想定)

ルータ間→SMF(数km～数百km想定、一旦SMF用のSFP+が足りなかったのでMMFで代用中)

#### ▼ 問題 1：SonicスイッチでMPLSが動かない

スイッチにMPLSの設定を入れてpingを送ってもパケットが届かなかった。  
tcpdumpで監視しても受信パケット0のままだった。

##### • 原因

Sonicに搭載されているBroadcomのスイッチチップ（ASIC）がLinuxカーネルに渡さず、チップ内部で直接転送する仕組みになっていた。

**想定：**パケット(PC)→Ethernet 0(スイッチ)→Linuxカーネル(スイッチ)→MPLS処理(スイッチ)→Ethernet 2(スイッチ)→転送

**実際：**パケット(PC)→Ethernet 0(スイッチ)→※チップがパケットを横取り→Ethernet 2 (スイッチ)→転送

※横取りのところでLinuxカーネルをスキップ、MPLSのルートを見ていない

##### • 試したこと

①Ethernetポートを別のnamespaceに移動→Sonicのドライバが root namespace以外にパケットを届けられなかった。

②Ethernet 0をrootに置き、ブリッジ+vethでnamespaceに引き込む→①と同じくドライバの制約でパケットが届かない、「Ethernet 0→bridge→Veth(virtual ethernet)」という経路はLinuxカーネルがパケットを受け取った時のみ機能する

③config interface ip add でスイッチにIPを設定しvethで接続→ARPとループバックは成功したが、パケットはチップがパケットを捕まえて転送した。Sonic自身のパケットはLinuxカーネルに届くが、TxからRxへ向かうなどの途中のパケットはASICがパケットを捕まえて転送してしまう。

- 結論と解決策

SonicではMPLSとQoSを扱うことは不可能と判断→SonicはL2スイッチとして使うだけにしてMPLSとQoSの処理はPC側のLinux namespaceで行う構成に切り替えた。

## ▼ 問題 2：問題 1 に起因するNIC(Network Interface Card)不足

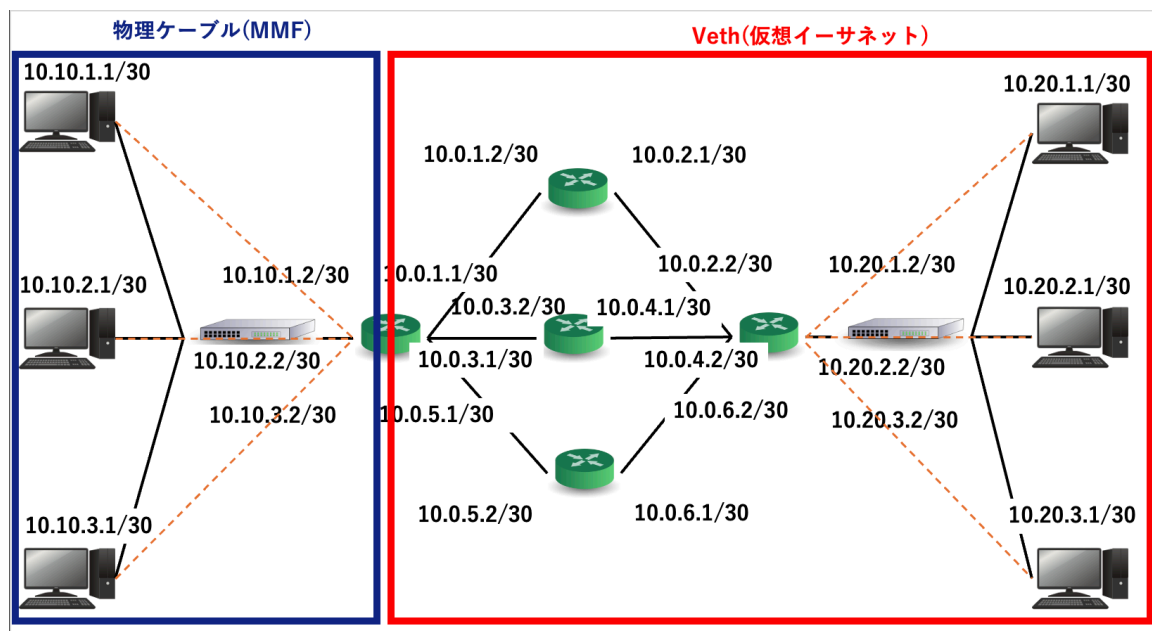
- 原因

当初SonicスイッチがMPLSの処理をし、PCはTxとRxを担当させる予定だった（TxとRxが計6つ、PCのNICも6つだったため問題ないと考えていた）。

しかし、問題 1 の解決策によってPC側でMPLSやQoSの設定をするようになった。

- 試したこと

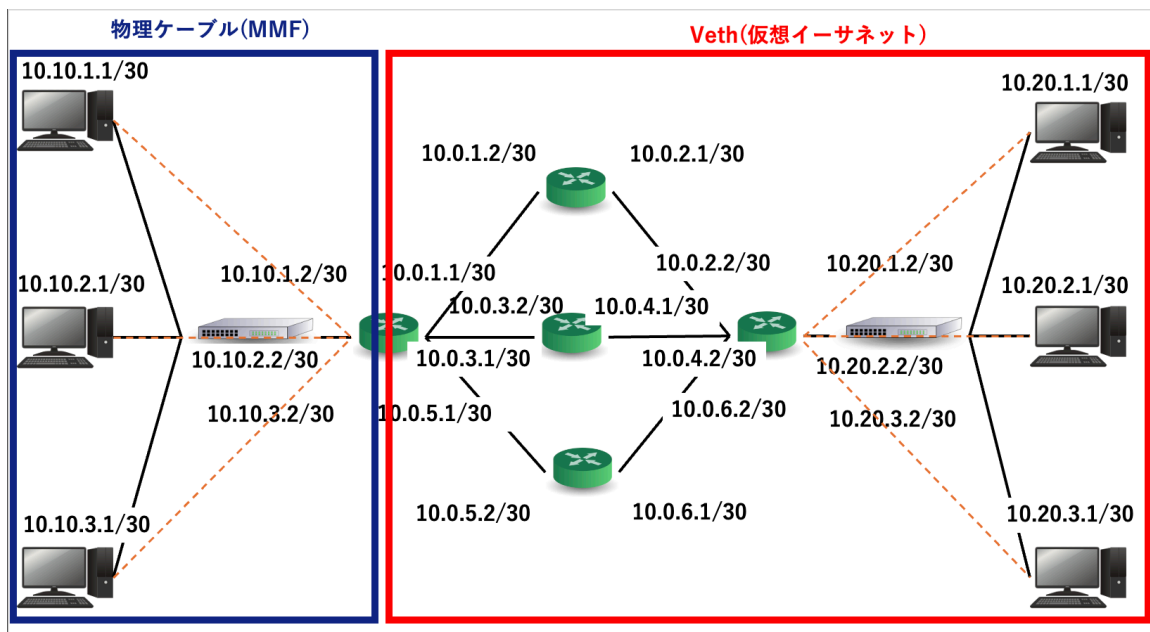
Rx分のNIC3個をPC側のLER\_Ingressに渡してMPLSとQoSの設定をするようになった。これによって先のノードにNICが残らなくなったため、Tx-LER\_Ingress間以外をVethで実装した。



### ▼ 実行コマンド

```
sudo bash scripts /measure.sh 60 normal
sudo bash scripts /measure.sh 60 failure
sudo bash scripts /measure.sh 60 failure_rsvp
git add -A
git commit -m "コメント"
git push origin main
```

### ▼ 実験方法、結果



- 基本は前回の進捗  [R8研究進捗報告\(2026.04.22\)](#) と条件を揃えることを意識した。
  - normal、failure、failure\_rsvpの3シナリオで測定  
failure、failure\_rsvpでは20s～40sにおいて障害を発生させている  
LER\_IngressとCoreRouter 1 の間で障害を発生させている。
  - rsvpの仕組み
- ① rsvp\_monitorがリンク障害を検知
- ▼ 検知の詳細(興味があれば)

## コンピュータ/sys/class/net/任意のNIC/operstate

上記のファイルはLinuxカーネルがリンクの状態を書き込むファイルとなっており、「ip link set 任意のリンク down」を行うとupがdownになる。  
rsvp\_monitor.shというファイルを作りこのupとdownを検知している。  
(検知のループは現在1秒)

### ▼ 検知の詳細 (Version2)

ip monitor link によってリンク変化時のみLinuxカーネルから通知を受け取る

変化がなければ待ち続ける (その間CPUはほぼ使わない)

## ②CoreRouter 1 を除いたルートを新たに構築

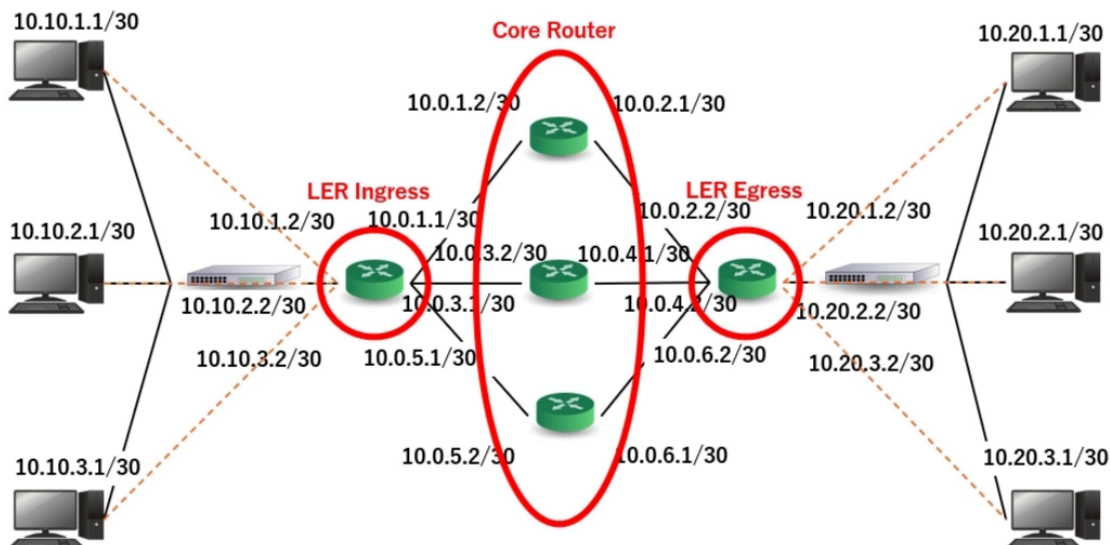
- 全シナリオで揃えている条件

| 条件       | 値                   |
|----------|---------------------|
| ネットワーク構成 | 上図                  |
| リンク帯域    | 100Mbps             |
| WRR重み    | 4:2:1               |
| 送信レート    | 500Mbps             |
| パケット設定   | UDP、1400B           |
| 計測時間     | 60s                 |
| クラス別遅延   | AF42:10ms AF43:40ms |

- 条件を変更する時のコマンド

```
sudo bash scripts/run.sh
```

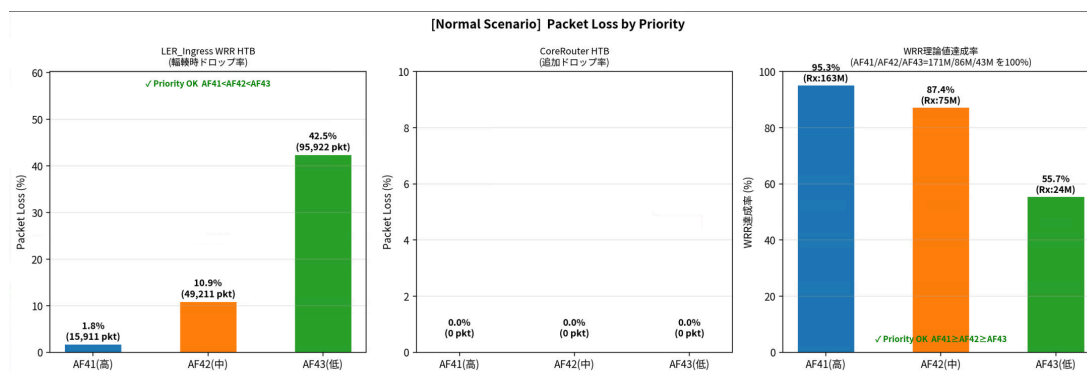
- パケロス測定位置



#### ▼ normalシナリオ

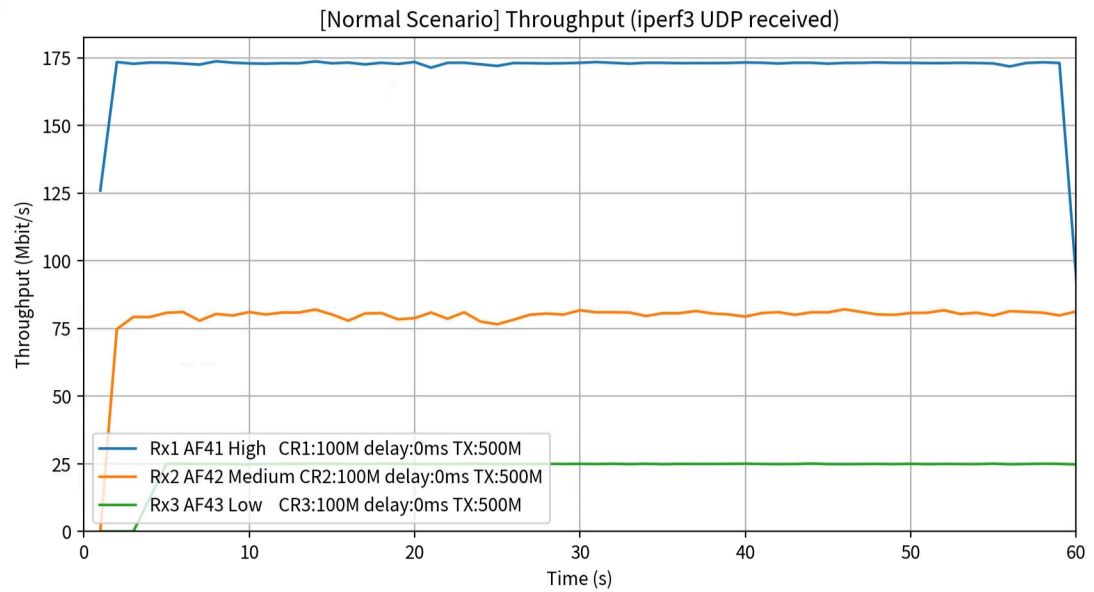
- 各ルーターにおけるパケットロスとWRR理論値達成率
- WRR理論値達成率：達成率 = 総帯域 ×  $\frac{\text{該当優先度の比率}}{\text{WRR比率の合計}}$

例) 高優先度：達成率 =  $300Mbps \times \frac{4}{4+2+1} = 171Mbps$

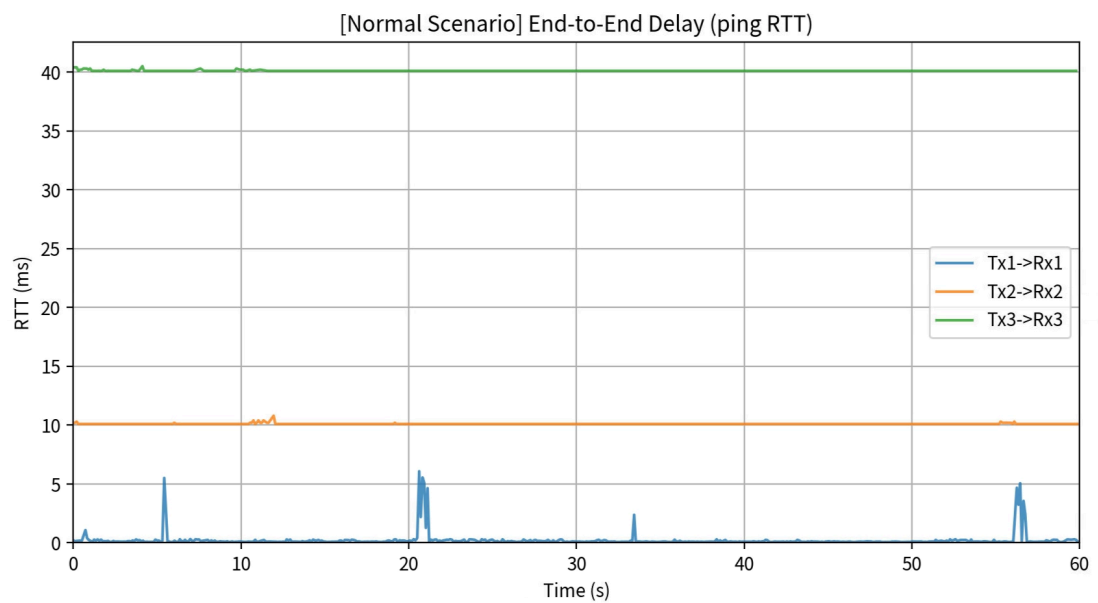


- スループット





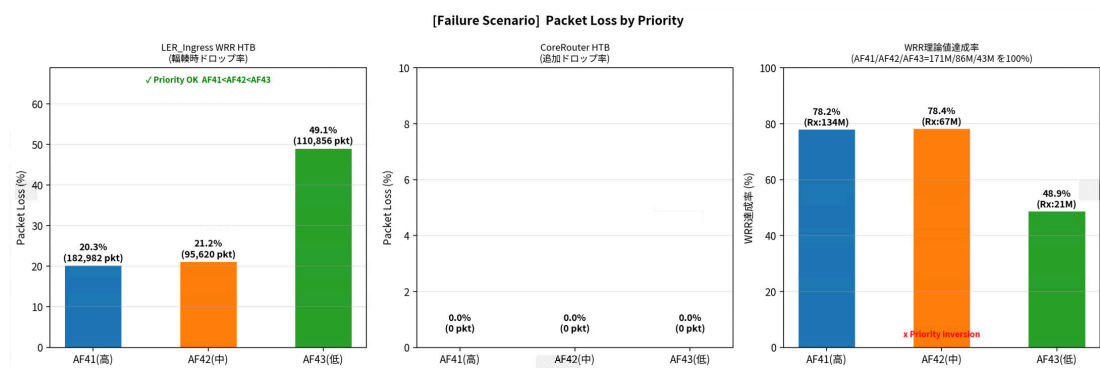
- end to end 遅延



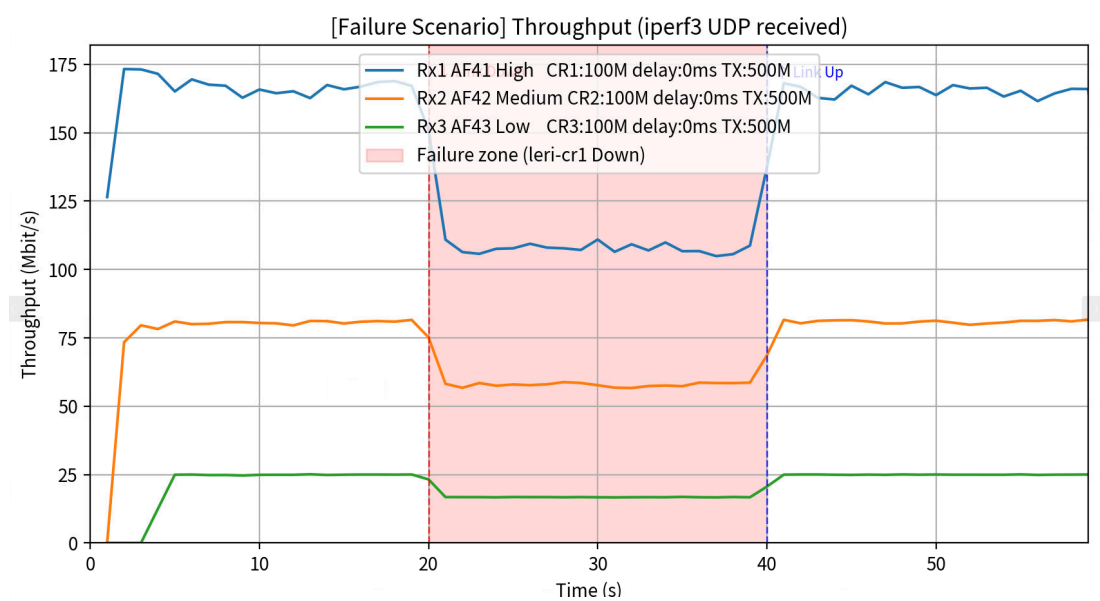
#### ▼ failureシナリオ

- 各ルーターにおけるパケットロスとWRR理論値達成率
- WRR理論値達成率：達成率 = 総帯域  $\times$   $\frac{\text{該当優先度の比率}}{\text{WRR比率の合計}}$

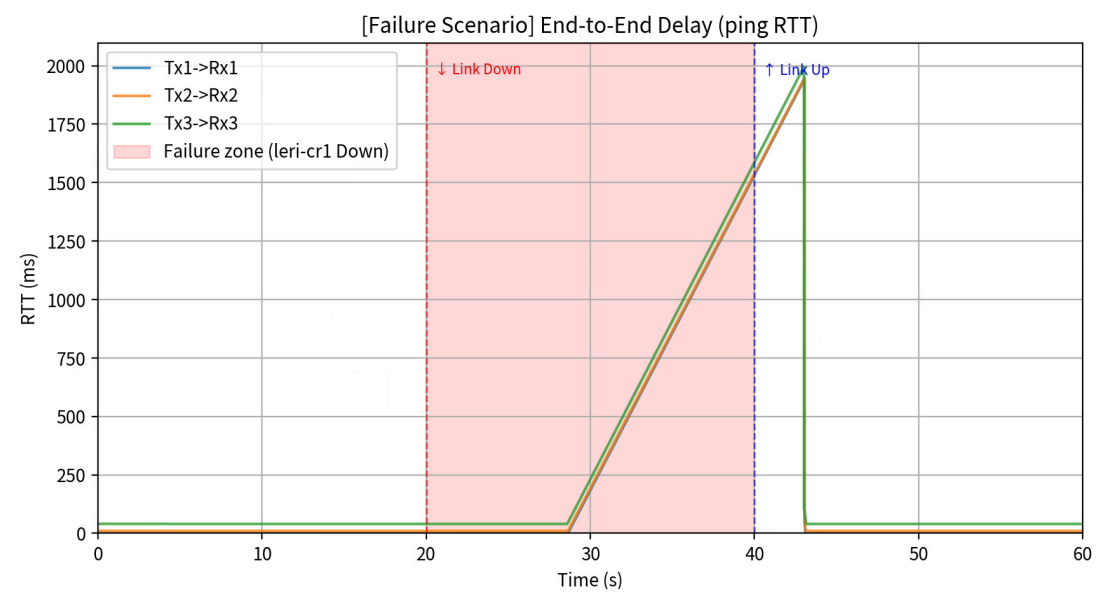
例) 高優先度：達成率 =  $300Mbps \times \frac{4}{4+2+1} = 171Mbps$



- スループット



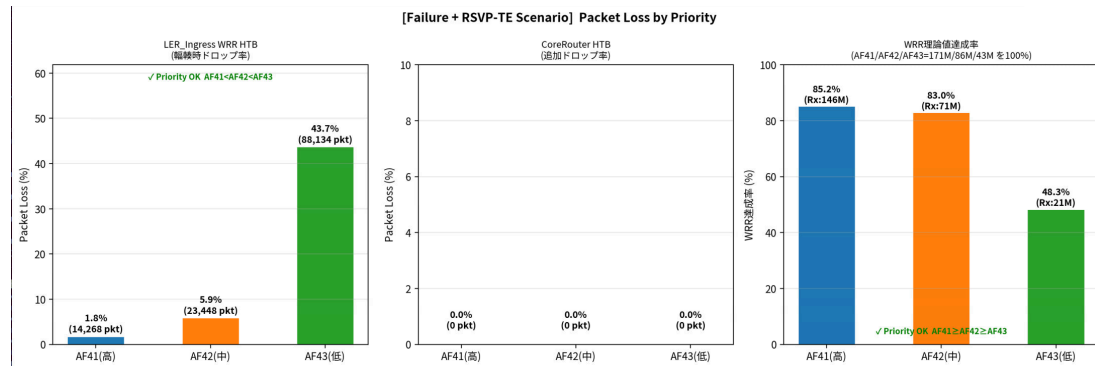
- end to end 遅延



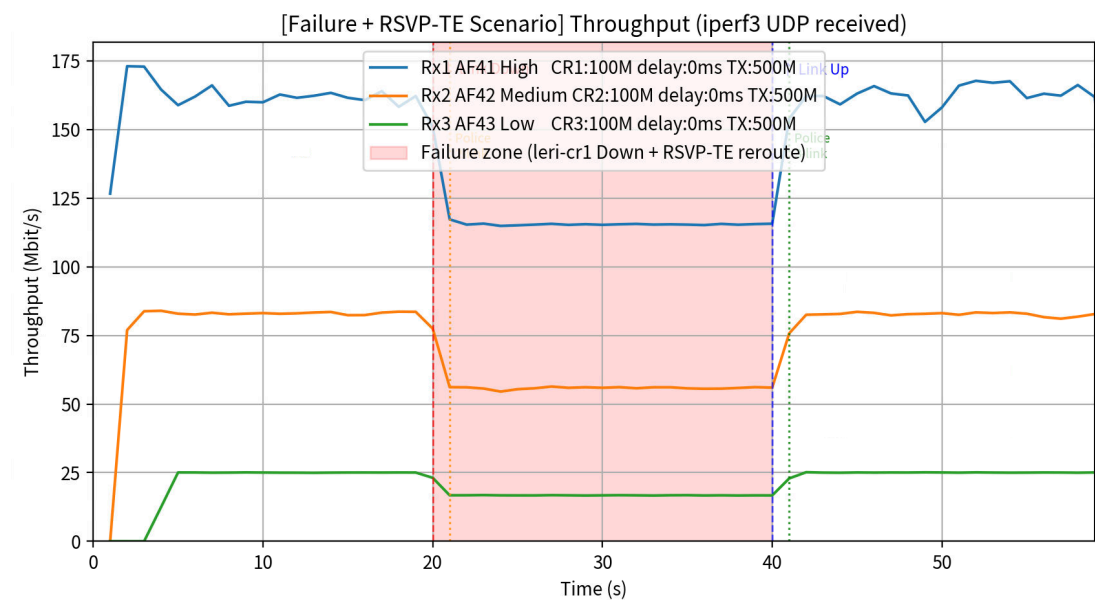
## ▼ failure\_rsvpシナリオ

- 各ルーターにおけるパケットロスとWRR理論値達成率
- WRR理論値達成率：達成率 = 総帯域 ×  $\frac{\text{該当優先度の比率}}{\text{WRR比率の合計}}$

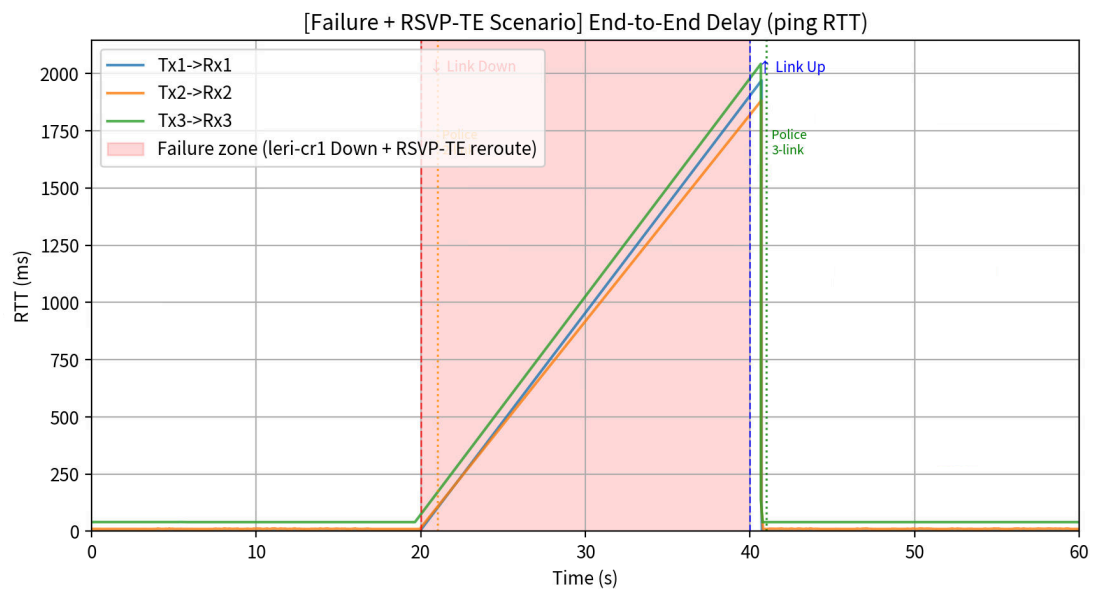
例) 高優先度：達成率 =  $300Mbps \times \frac{4}{4+2+1} = 171Mbps$



- スループット

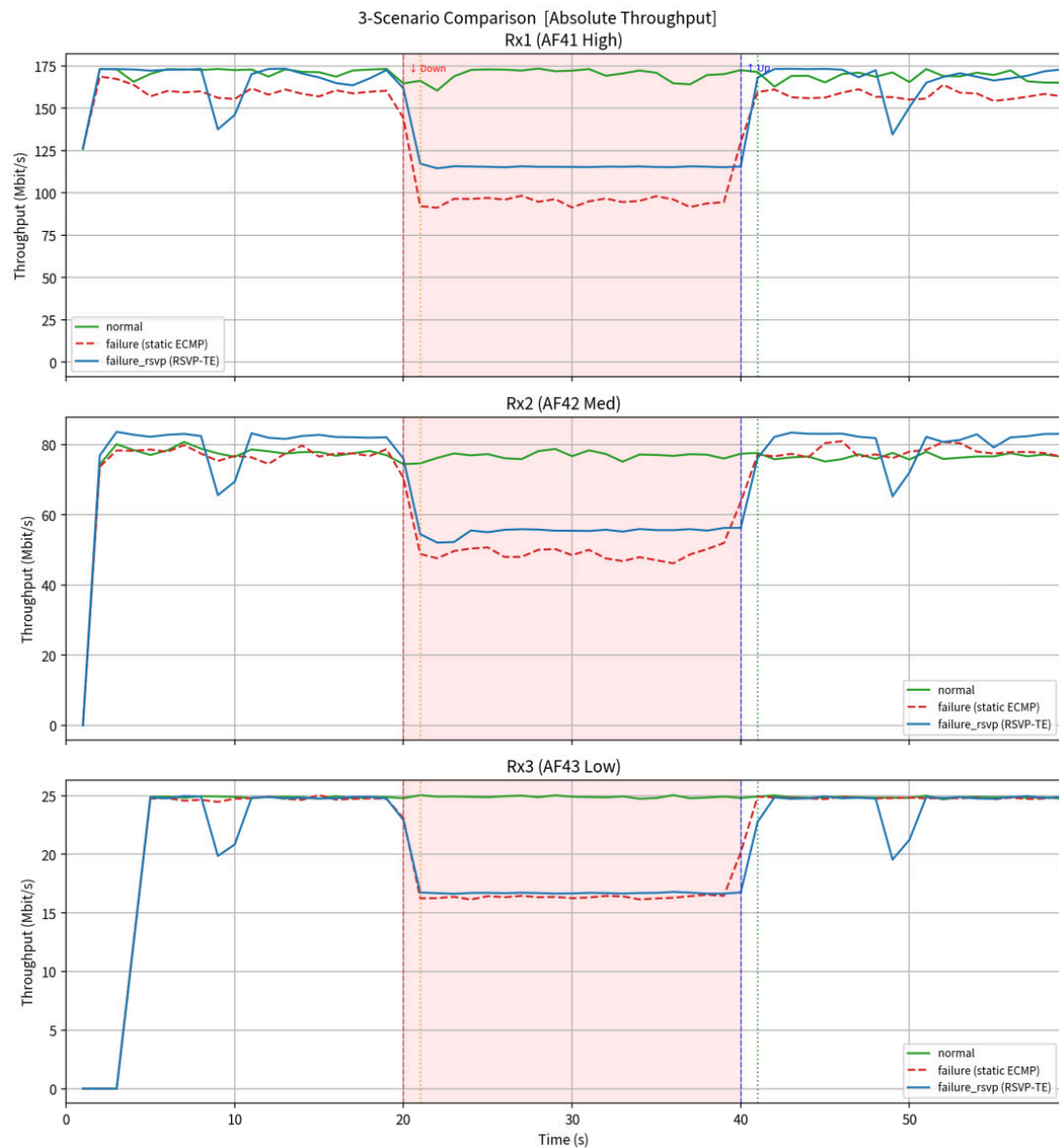


- end to end 遅延

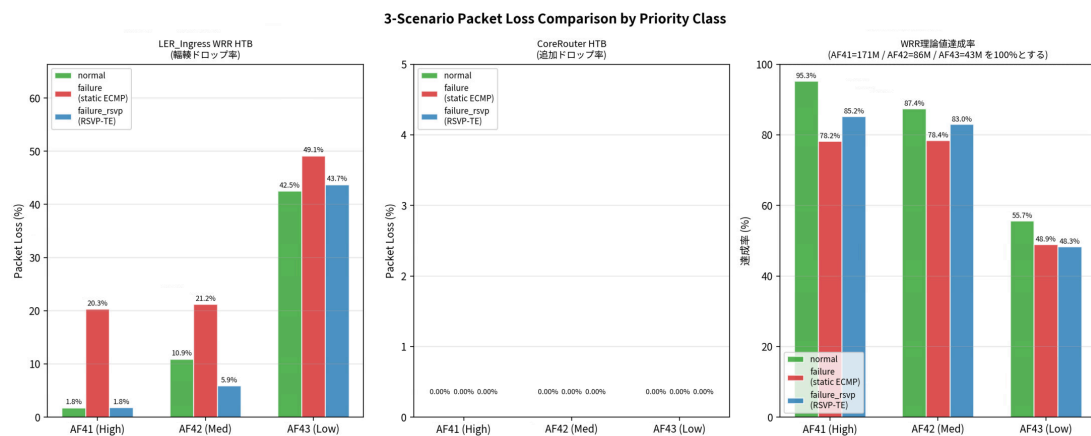


- 3 シナリオ間の比較

スループット

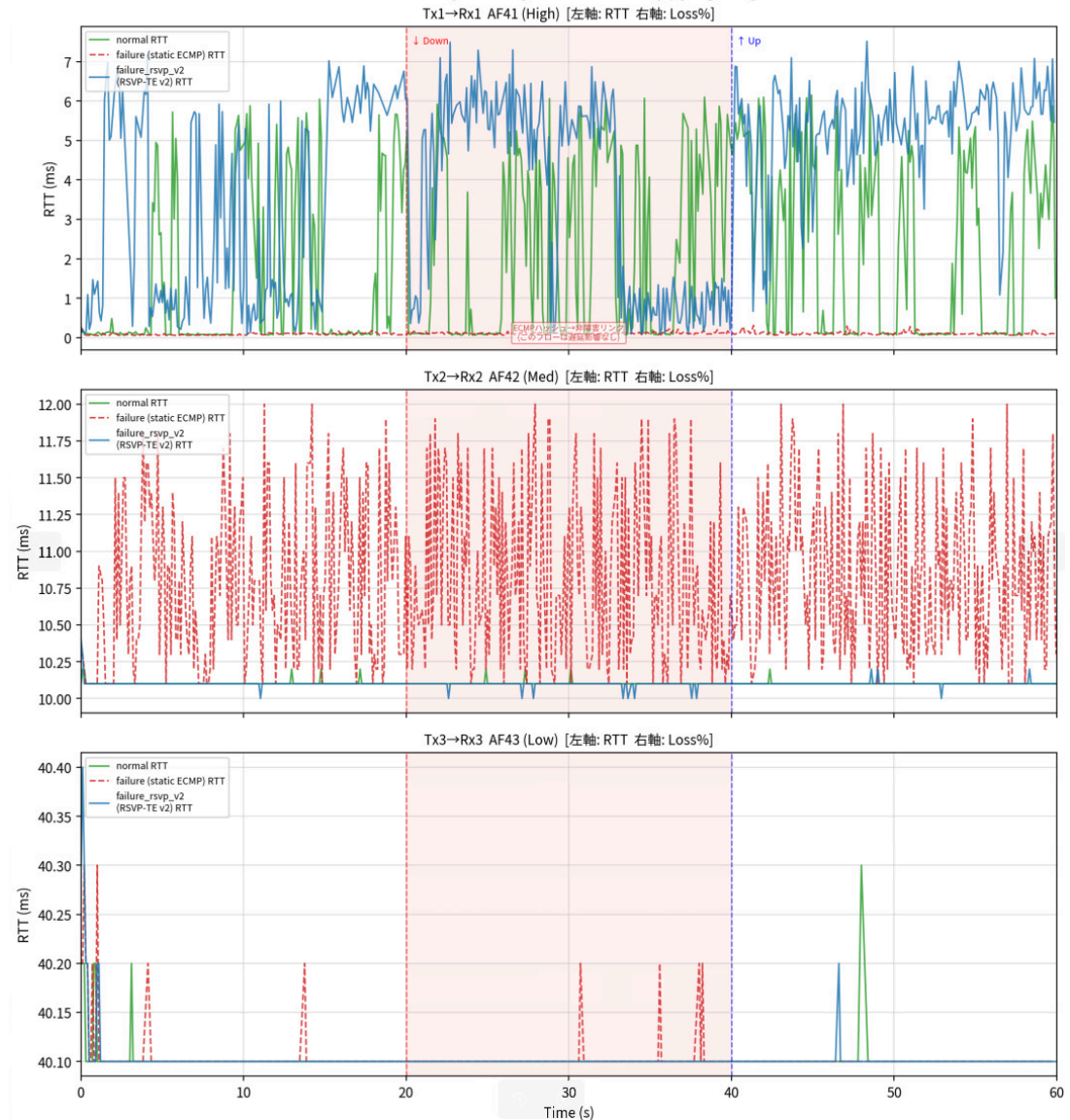


## 各ルーターにおけるパケットロス



## end to end 遅延

### 3-Scenario Comparison [End-to-End Delay / ping RTT]



- スループットについて

failureとfailure\_rsvp間で障害中にRx1(高)：約95Mbps→約120Mbps  
 Rx2(中)：約50Mbps→約55Mbpsの改善が見られた。

- パケットロスについて

failureとfailure\_rsvp間でRx1(高)：20.3%→1.8% Rx2(中)：21.2%→5.9%  
 Rx3(低)：49.1%→43.7%

の改善が見られた。LER\_Ingressで供給過多となったパケットは既にドロップしているためCoreRouter内における新たなパケットのドロップはいずれのシナリオでも見られなかった。

- WRR達成比率について

failureとfailure\_rsvp間で、Rx1(高)：78.2%→85.2%(100%=171Mbps)  
Rx2(中)：78.4%→83%(100%=86Mbps)の改善が見られた。一方で  
Rx3(低)：48.9%→48.3%(100%=43Mbps)と微減しており、ここに関しては  
効果が見られなかった。

- end to end 遅延について

failureシナリオのRx1において、遅延がほぼ0msで推移しており、直観に反する結果となった。おそらく何らかの設定ミスだと考えているが、原因不明なため究明中。

→ failureとfailure\_rsvpで障害発生の仕方が異なっていることが問題？

failure：リンクはupのままパケットは即座にドロップ、これによりキューは常に空いたの状態なため遅延は0と計測されてしまう？

failure\_rsvp：20s時点でリンクをdown、CoreRouter 2、3のみを使ったルートが再構築されるため遅延が計測される？

(障害発生の方を揃えたかったが、リンクをdownさせるとLinuxカーネルが勝手に迂回ルートを作る仕様が合ったため上記のような障害の模擬を行った。)

## まとめ

- 今回の検証の結果からは前回の進捗報告  R8研究進捗報告(2026.04.22) におけるOMNeT++でのシミュレーション時と大きな齟齬は見られず、「障害発生を検知して別ルートに迂回させつつ、輻輳している環境でも全体の通信の品質を保つ」という当初の目的をおおむね達成出来ているのではないかと考える。
- まだスイッチの仕様(チップの仕様)を把握しきれていなかったり、数kmから数百km想定の部分もMMFで代用している、Gbpsオーダーは未検証など詰め切れていない部分は数多くあるので1個ずつ実機検証の精度を上げていきたい。

## • Next Steps

- end to end 遅延のグラフに対する考察
- 送信レートと総帯域を上げた時の挙動検証(Gbpsオーダー)
- RSVPモニターの精度検討(観測頻度と負荷の大きさのつり合い等)
- スイッチ側のLinuxにDockerを立ててSonicのコマンドで入れる、スイッチだけで完結できる、2台繋いで実験する、PC側ではQoSのフラグをたてるだけ
- ルーティングマップの確認

## ▼ Memo

保守

備忘録1

備忘録2

備忘録3

Github

- Devices and Equipment Used (including name and model)

## ▼ PC specifications

Device name Satokilchikawa

Processor AMD Ryzen 5 5560U with Radeon Graphics 2.30 GHz

Installed RAM 16.0 GB (15.4 GB usable)

Device ID AD1E026E-44EF-407C-A8BE-29656F72170B

Product ID 00342-22119-31496-AAOEM

System type 64-bit operating system, x64-based processor

Pen and touch Pen input and touch input are not available for this display.

Edition Windows 11 Home

Version 22H2

Install date 2023/03/13

OS build 22621.2283

Experience Windows Feature Experience Pack 1000.22662.1000.0

## ▼ Mini PC specifications

Device name DESKTOP-00PTPEG

Processor Intel(R) N150 (800 MHz)

Installed RAM 8.00 GB (7.78 GB usable)

Device ID 89860565-1BA3-4E0E-979C-E9BE7463E16E

Product ID 00330-80000-00000-AA161

System type 64-bit operating system, x64-based processor

Pen and touch Pen input and touch input are not available for this display.

VirtualBox-7.2.4-170995-Win

ubuntu-24.04.3-desktop-amd64

## • Notes

~~OMNet++ 6.0.1 IDE omnetpp-6.1rc1-windows-x86\_64~~



OMNeT++ 6.1 IDE

チュートリアルはサンプルコードを参考に挙動を確かめた

## References

- ① [Introduction - OMNeT++ Technical Articles \(omnetpp.org\)](#).
- ② [OMNeT++ - Simulation Manual \(omnetpp.org\)](#).
- ③ [https://www.fit.ac.jp/~oida/download/exp/pkswitch\\_net/ide.html#gui](https://www.fit.ac.jp/~oida/download/exp/pkswitch_net/ide.html#gui)
- ④ [OMNeT++ Simulation Library: cenvir.h Source File \(omnetpp.org\)](#).
- ⑤ [INET Framework - INET Framework](#)